

Memory Management

subtitle

1s2026

Gabriela Bittencourt, João Pedro Leôncio and Vinícius Peixoto

Contents



1	Understanding Resources	3
1.a	Memory x Storage	4
1.b	Program x Process	5
1.c	Memory Management Historical Advance	6
1.d	Non-contiguous Memory Allocation	7
2	Physical Memory	8
2.a	Limited resource	9
3	Virtual Memory	10
3.a	Motivation	11
4	Paging	12
4.a	Page Table Entries (PTE)	13
4.b	Levels	14
4.c	Linux Kernel implementation/design	15
5	Struct	16
6	References	17
6.a	References	18

1 Understanding Resources

Memory x Storage



When we talk about ‘memory’ in OS context, we are talking about **RAM**

Memory is NOT Storage

- Memory
 - fast, expensive, volatile
- Storage
 - slow, cheap, consistent

Program x Process



- Program
 - Code. Static entity.
 - Needs storage
 - Example: Your C code from MC202: the .c file
- Process
 - The execution of the code. Dynamic entity.
 - Needs memory: the instructions will be load in specific space of memory (), variables will be set to specific spaces in memory ()
 - Example: The instance of your text editor while you are writing a program; the instance of the terminal running when you are navigating through it to compile your code and when you run it; the instance of your program during the short period of time your computer is executing your code

Memory Management Historical Advance



- Contiguous Memory Allocation (no abstraction of memory)
 - One part is reserved to the OS, the other runs a single process
 - One process at the time: in order to change the process executed, the OS must save the previous process in disk and then load the next one.
 - Simple memory management
 - Poor memory utilization
- Non-contiguous Memory Allocation
 - a

Non-contiguous Memory Allocation

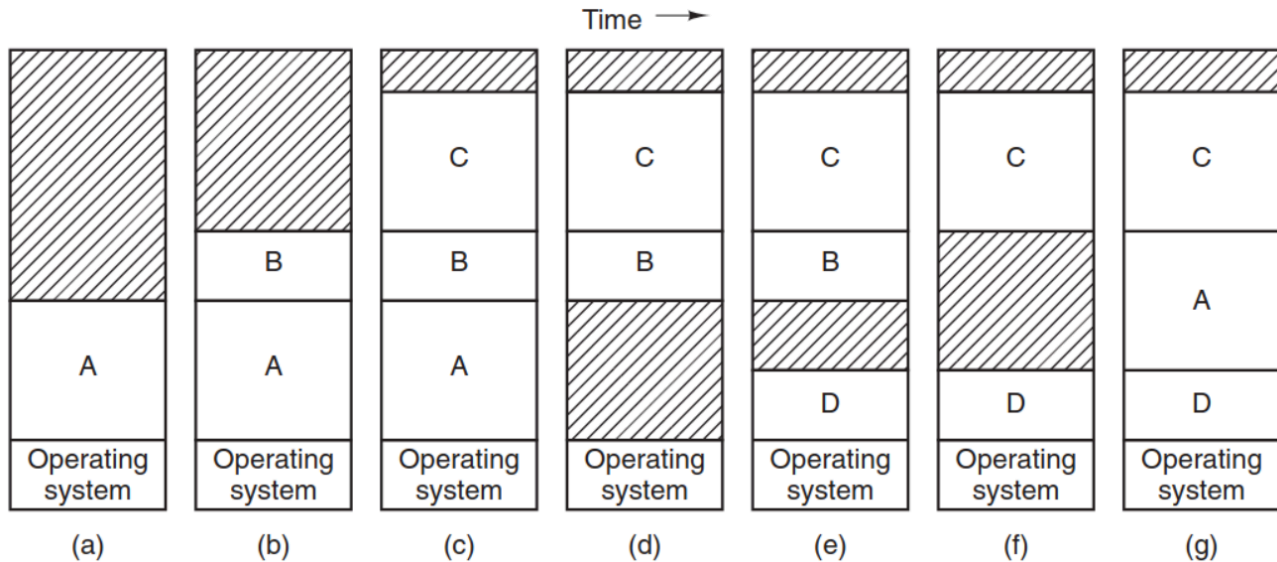


Figure 1: Memory allocation of 5 processes (programs: A, B, C, D and A again) over time.

2 Physical Memory

Limited resource



```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	14Gi	12Gi	803Mi	4.5Gi	6.2Gi	2.7Gi
Swap:	4.0Gi	2.6Gi	1.4Gi			

3 Virtual Memory

Motivation



- Resource management
 - Ideally all processes would have access to the full memory at the moment of execution
 - In reality, each process has a access to a limited amount of memory that can expand and shrink depending on demand
- Scope security
 - security of data from different processes

4 Paging

Page Table Entries (PTE)



pte

Levels



pte is cool and all, but it's not enough

Linux Kernel implementation/design

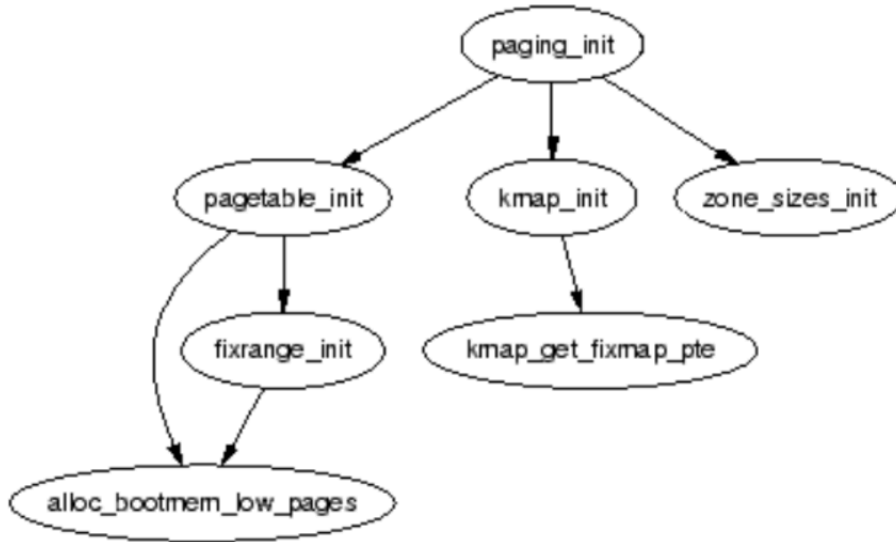


Figure 2: a - <https://www.kernel.org/doc/gorman/html/understand/understand006.html>

5 Struct

6 References

References



- Tanenbaum, Andrew. (2012) '3. Memory Management', in *Modern operating systems*. Pearson Education, Inc., 4th ed.
- TODO: add kernel codes
- TODO: add kernel docs